

Arquitetura e Engenharia de Dados: Aplicativo de Controle Financeiro (Cofrinho)

Este documento descreve as decisões arquiteturais, o modelo de banco de dados e o motor de funcionamento (Financial Processing Engine) do software de controle financeiro "Cofrinho", desenhado sob uma ótica de Engenharia de Dados.

1. Visão Geral da Arquitetura (Tech Stack)

A aplicação foi construída sobre uma arquitetura moderna e serverless (desacoplada), focando em baixa latência e alta escalabilidade:

- Frontend (Client-side):** Desenvolvido em **React.js** com a framework **Next.js (App Router)**. A renderização é desenhada para um comportamento *Single Page Application (SPA)* responsiva, embalada como um aplicativo (PWA).
- Estilização:** Vanilla CSS interligado com **TailwindCSS** para abstração de design tokens (Design System unificado).
- Backend as a Service (BaaS):** O trâmite em tempo real, armazenamento de dados relacionais e sistema de autenticação forte roteiam unicamente no ecossistema do **Supabase** (PostgreSQL na base).
- Hospedagem & CI/CD:** **Vercel** Cloud Edge Network.
- Micro-integrações:** Webhooks configurados para comunicação bidirecional com a plataforma **Twilio** (WhatsApp API).

2. Modelagem Relacional (Entity Relationship)

O banco relacional Postgres possui entidades unificadoras para não pulverizar os relatórios globais de fluxo de caixa em tabelas satélites.

Entidade Central: `transactions`

Esta é a fonte da verdade de caixa (Ledger Table). Centralizamos **diferentes tipos de obrigações financeiras** em uma tabela única diferenciada pelo atributo `type`:

Coluna	Tipo	Descrição
`id`	UUID	PK - Identificador unificado da transação.
`user_id`	UUID	FK -> auth.users (Garante RLS / Isolação Tental).
`amount`	NUMERIC	Valor bruto da transação financeira.
`description`	VARCHAR	Rótulo customizável de consumo.
`category`	VARCHAR	Taxonomia para agrupamento/agregação.
`type`	ENUM	Regra de negócio limitadora: `income` (Receitas), `expense` (Débitos à vista), `credit` (Cartões de crédito), `fixed` (Gastos fixos mensais).
`date`	DATE	Data matriz (Y-M-D) que a transação <i>ocorreu</i> fisicamente.
`is_paid`	BOOLEAN	Controla o status de provisionamento vs. liquidação real no caixa.
`credit_card_id`	UUID	FK -> credit_cards (Nullable - Utilizado quando `type` = `credit`).

Entidade Acessória: `credit\_cards`

Responsável por orquestrar janelas de faturamento temporal:

- `limit\_amount` (NUMERIC): Teto de gastos.
- `closing\_day` (INT) & `due\_day` (INT): Âncoras espaciais que decidem para qual "mês contábil" as transações atreladas à `transactions` irão flutuar.

3. O "Motor Financeiro" (Data Transform & Fluxo de Caixa)

O gargalo de muitos sistemas financeiros é o acoplamento do Cartão de Crédito. No Cofrinho, atuamos com Data Shifting (Projeção Financeira Temporal).

Lógica de Faturamento Diferido (Invoice Logic)

Se o usuário lança um débito em *25 de Março*, mas o cartão "Nubank" corta no dia 20 e vence no dia 4 do mês seguinte:

- O Front-End renderiza o **Histórico** lendo a coluna física `date` (Março).
- O **Motor de Faturas** (`getInvoicePeriod`) intercepta a leitura no Dashboard e muda visualmente o destino da compra: Se (Data de Compra > Closing_Day), injeta a despesa na contagem do mês de Abril.

Algoritmo Regressivo de Limites de Cartão

Em vez de depender de datas, a arquitetura do "Consumo do Cartão" depende restritamente da prova da liquidação da dívida:

O Limite utilizado é a soma global da tabela `transactions` onde:

``FK` = credit_card AND `is_paid` == FALSE`

(Isso recria computacionalmente como os bancos processam faturas atrasadas sem quebrar as linhas do tempo de relatórios retroativos).

4. Workflows e Jobs Secundários

Motor Híbrido de Busca Rápida

A página de listagem possui um Search Client-Side que permite busca global de todos os meses, mapeando não só *Strings* (`description/category`), mas realizando uma conversão dinâmica de tipos de float para String (com *parsing* interligado de pontos e vírgulas) garantindo hits nos decimais. E.g., pesquisar *"50.00"* encontra transações gravadas como *`50`* no postgres.

Gateway Twilio via WhatsApp Route

Existe uma infraestrutura separada de ingestão de dados em tempo real.

- Caminho:** Mensagem de usuário -> API do Twilio Webhook -> Node.js `app/api/whatsapp/route.ts`.
- Inteligência de ETL:** O código Node captura o dado cru em formato String (*"55.90 lFood Alimentação Nubank"*), dispara uma Regex interna que separa [Valor], extrai [Descrição] e [Categoria], efetua uma varredura paralela por cartões do usuário para converter automaticamente os *flags* relacionais (assumindo `type` e a FK do cartão), aciona a `SUPABASE_SERVICE_ROLE_KEY` (bypassing RLS) para realizar a injeção do JSON na entidade destino e devolve TwiML XML via Webhook garantindo a confiabilidade transacional.

5. Segurança do Pipeline de Dados

- **Proteções Passivas:** Nenhuma transação pode ser vazada pois as políticas RLS (*Row Level Security*) do PostgreSQL foram amarradas com força à chave `'uid()'` instanciada nos JWTs do Vercel Client Wrapper.
- Todas as operações em massa da UI usam Mutações de Banco seguida de reconstrução assíncrona do cache da página local para sensação de zero-latência.